```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings('ignore')


sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
df = pd.read_csv('/content/StudentPerformanceFactors.csv')
df.head()
```

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previo
0	23	84	Low	High	No	7	
1	19	64	Low	Medium	No	8	
2	24	98	Medium	Medium	Yes	7	
3	29	89	Low	Medium	Yes	8	
4	19	92	Medium	Medium	Yes	6	

Next steps: Generate code with df | New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6607 entries, 0 to 6606
Data columns (total 20 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Hours_Studied               6607 non-null   int64
 1   Attendance                  6607 non-null   int64
 2   Parental_Involvement        6607 non-null   object
 3   Access_to_Resources         6607 non-null   object
 4   Extracurricular_Activities  6607 non-null   object
 5   Sleep_Hours                 6607 non-null   int64
 6   Previous_Scores             6607 non-null   int64
 7   Motivation_Level            6607 non-null   object
 8   Internet_Access             6607 non-null   object
 9   Tutoring_Sessions           6607 non-null   int64
 10  Family_Income               6607 non-null   object
 11  Teacher_Quality             6529 non-null   object
 12  School_Type                 6607 non-null   object
 13  Peer_Influence              6607 non-null   object
 14  Physical_Activity           6607 non-null   int64
 15  Learning_Disabilities       6607 non-null   object
 16  Parental_Education_Level    6517 non-null   object
 17  Distance_from_Home          6540 non-null   object
 18  Gender                      6607 non-null   object
 19  Exam_Score                  6607 non-null   int64
dtypes: int64(7), object(13)
memory usage: 1.0+ MB
```

```
df.describe()
```

	Hours_Studied	Attendance	Sleep_Hours	Previous_Scores	Tutoring_Sessions	Physical_Activity	Exam_Score
count	6607.000000	6607.000000	6607.00000	6607.000000	6607.000000	6607.000000	6607.000000
mean	19.975329	79.977448	7.02906	75.070531	1.493719	2.967610	67.235659
std	5.990594	11.547475	1.46812	14.399784	1.230570	1.031231	3.890456
min	1.000000	60.000000	4.00000	50.000000	0.000000	0.000000	55.000000
25%	16.000000	70.000000	6.00000	63.000000	1.000000	2.000000	65.000000
50%	20.000000	80.000000	7.00000	75.000000	1.000000	3.000000	67.000000
75%	24.000000	90.000000	8.00000	88.000000	2.000000	4.000000	69.000000
max	44.000000	100.000000	10.00000	100.000000	8.000000	6.000000	101.000000

```
df.drop(index = df[df['Exam_Score'] > 100].index , inplace = True , axis = 0 )
categorical = df.select_dtypes(include = ['object' , 'category']).columns.tolist()
for col in categorical :
    print('\n' , df[col].value_counts())
```

```
No     2669
Name: count, dtype: int64

 Motivation_Level
Medium    3351
Low       1936
High      1319
Name: count, dtype: int64

 Internet_Access
Yes    6108
No      498
Name: count, dtype: int64

 Family_Income
Low       2672
Medium    2666
High      1268
Name: count, dtype: int64

 Teacher_Quality
Medium    3925
High      1946
Low        657
Name: count, dtype: int64

 School_Type
Public     4597
Private    2009
Name: count, dtype: int64

 Peer_Influence
Positive    2637
Neutral     2592
Negative    1377
Name: count, dtype: int64

 Learning_Disabilities
No     5911
Yes     695
```

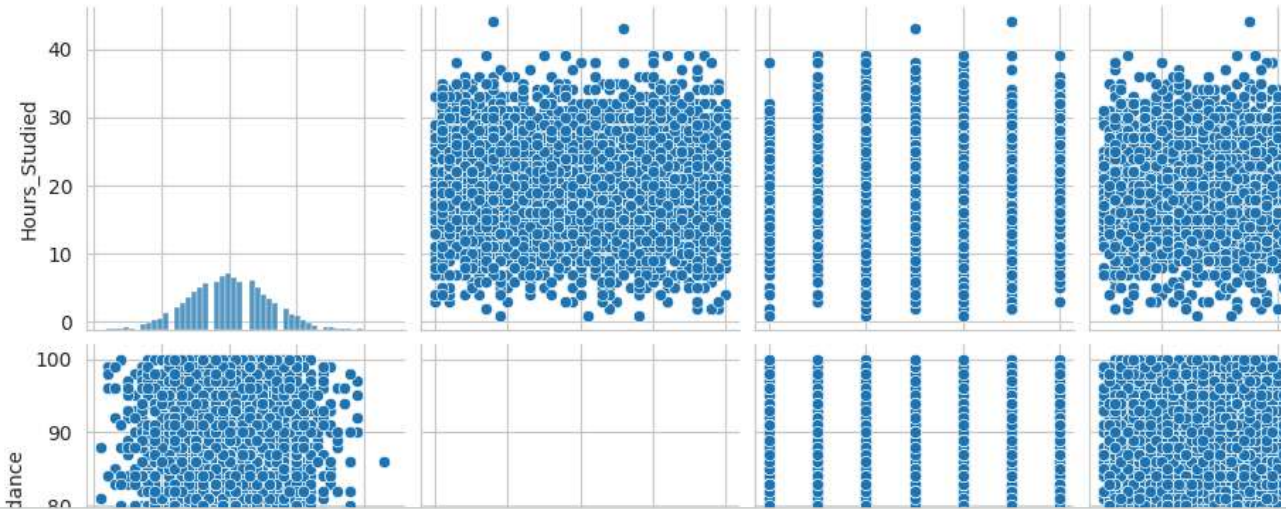
```
Male      3814
Female    2792
Name: count, dtype: int64
```

```
categorical = df.select_dtypes(include = ['object' , 'category']).columns.tolist()
for col in categorical :
    print('\n' , df[col].value_counts())
```

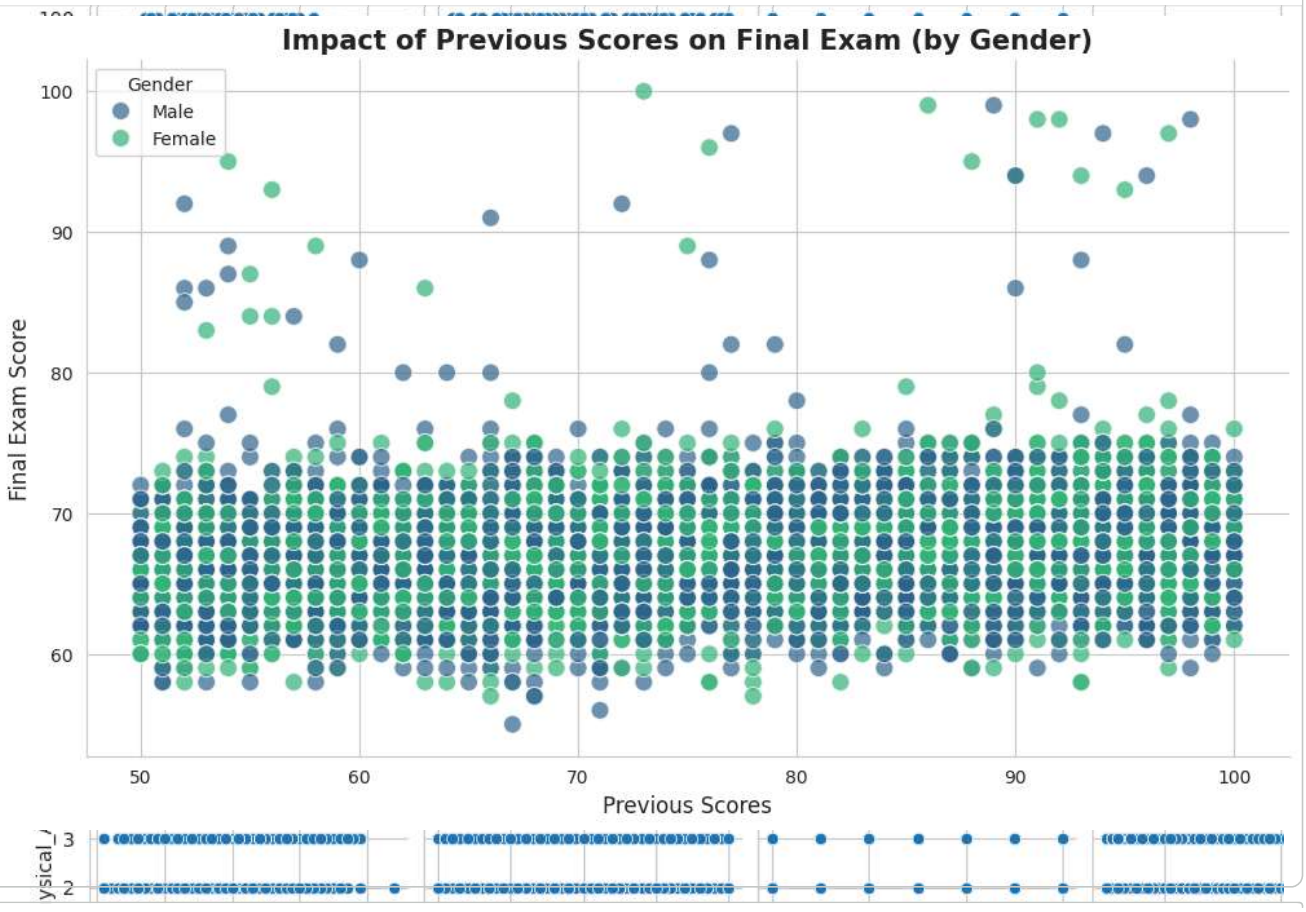
```
No     2669
Name: count, dtype: int64

 Motivation_Level
Medium    3351
Low       1936
High      1319
Name: count, dtype: int64

 Internet_Access
Yes    6108
No      498
Name: count, dtype: int64

 Family_Income
Low       2672
Medium    2666
High      1268
Name: count, dtype: int64

 Teacher_Quality
Medium    3925
High      1946
Low        657
Name: count, dtype: int64

 School_Type
Public     4597
Private    2009
Name: count, dtype: int64

 Peer_Influence
Positive    2637
Neutral     2592
Negative    1377
Name: count, dtype: int64

 Learning_Disabilities
No     5911
Yes     695
Name: count, dtype: int64

 Parental_Education_Level
High School     3222
College         1989
Postgraduate    1305
Name: count, dtype: int64

 Distance_from_Home
Near        3884
Moderate    1997
Far          658
Name: count, dtype: int64

 Gender
Male      3814
Female    2792
Name: count, dtype: int64
```

```
sns.pairplot(data = df )
```

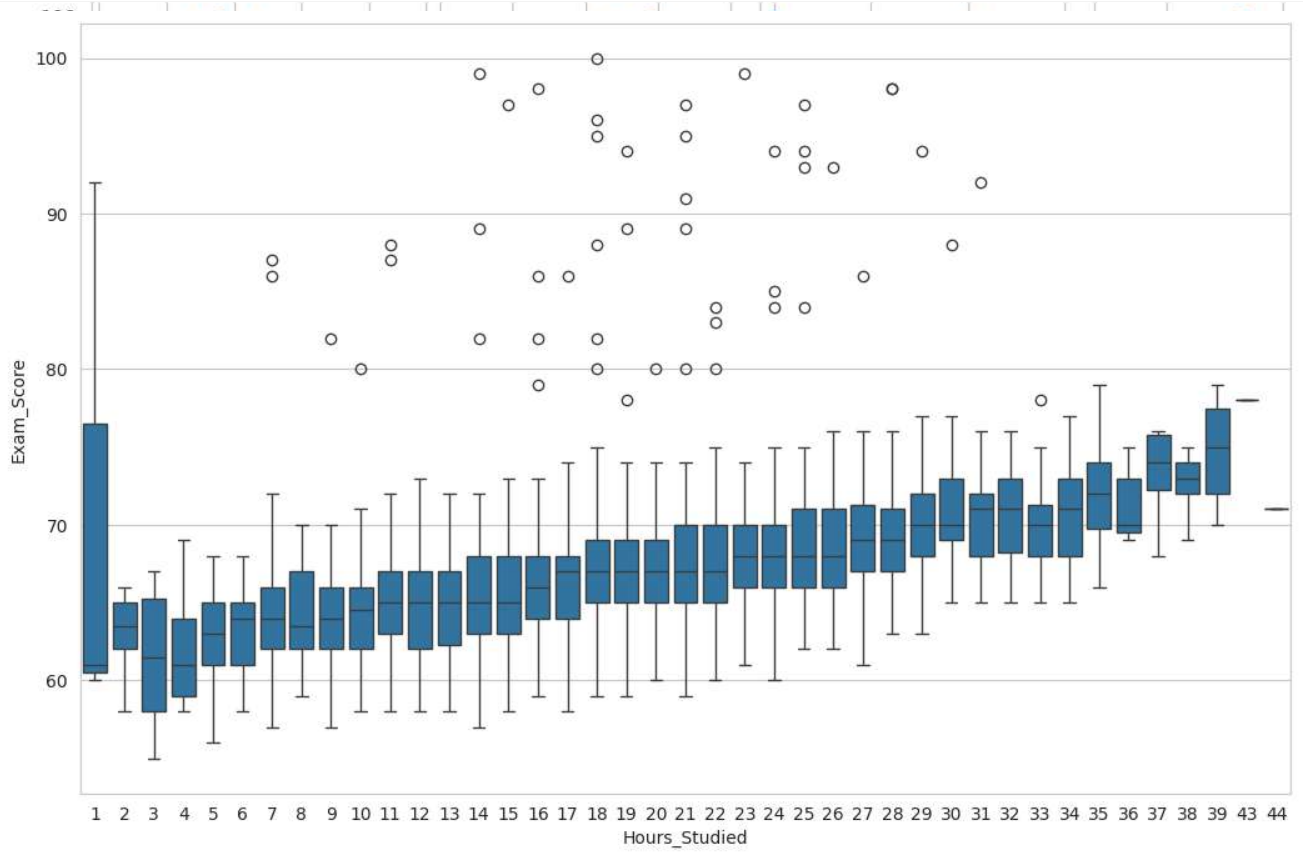
```
<seaborn.axisgrid.PairGrid at 0x7a7d250b5eb0>
```




```
plt.figure(figsize=(12, 7))
sns.scatterplot(
    data=df,
    x='Previous_Scores',
    y='Exam_Score',
    hue='Gender',
    palette='viridis',
    alpha=0.7,
    s=100
)

plt.title("Impact of Previous Scores on Final Exam (by Gender)", fontsize=15, fontweight='bold')
plt.xlabel("Previous Scores", fontsize=12)
plt.ylabel("Final Exam Score", fontsize=12)
plt.legend(title='Gender', loc='upper left')
sns.despine()
plt.show()
```



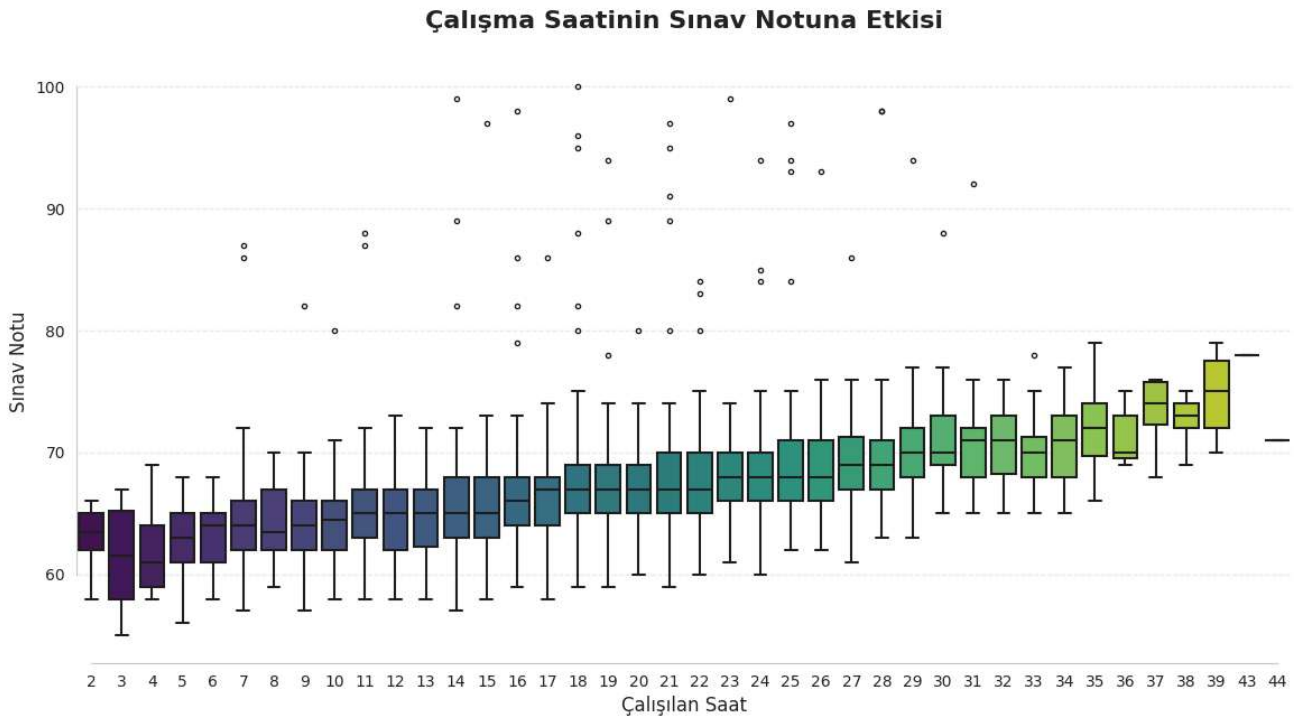
```
plt.figure(figsize = (12.5,8))
sns.boxplot(data = df , x = 'Hours_Studied' , y = 'Exam_Score')
plt.show()
```



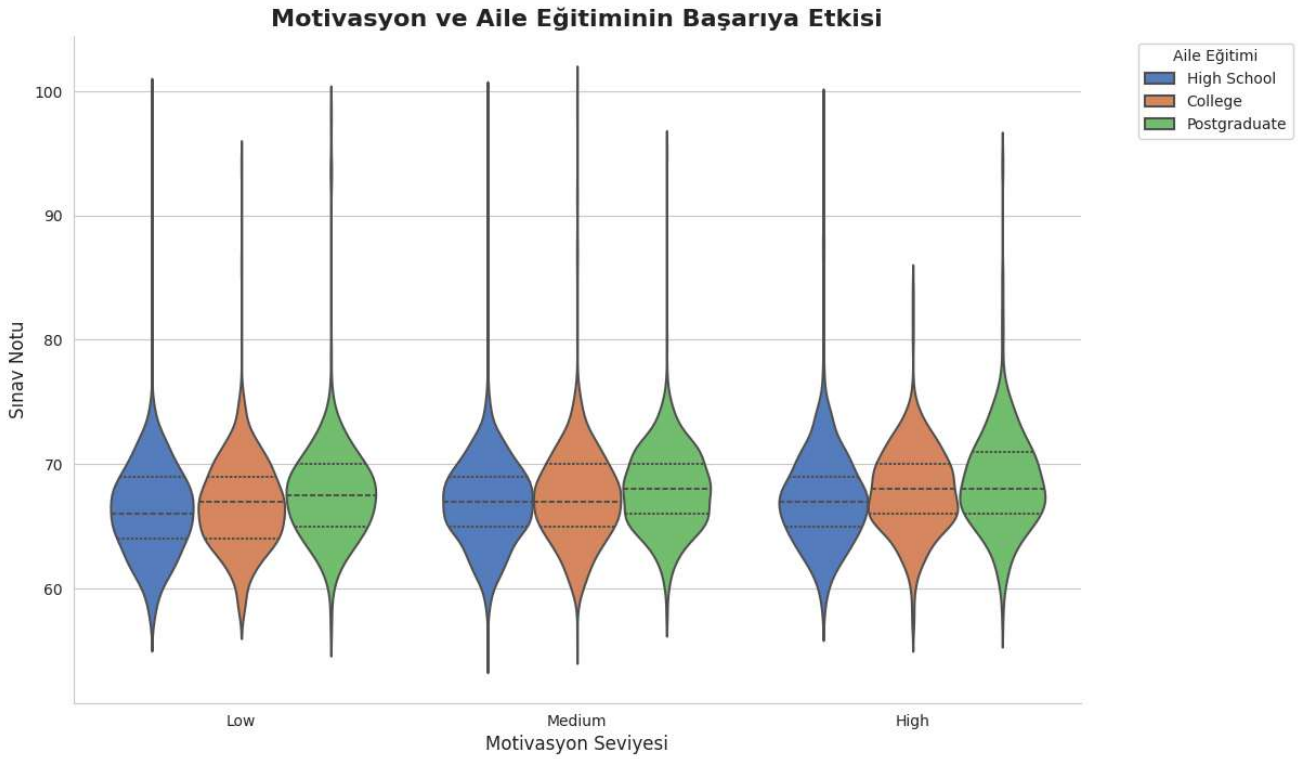

```
df[df['Hours_Studied'] == 1]
```

	Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Pre
262	1	69	High	Medium	Yes	6	
4725	1	81	Medium	Medium	Yes	8	
4779	1	88	Medium	High	Yes	4	

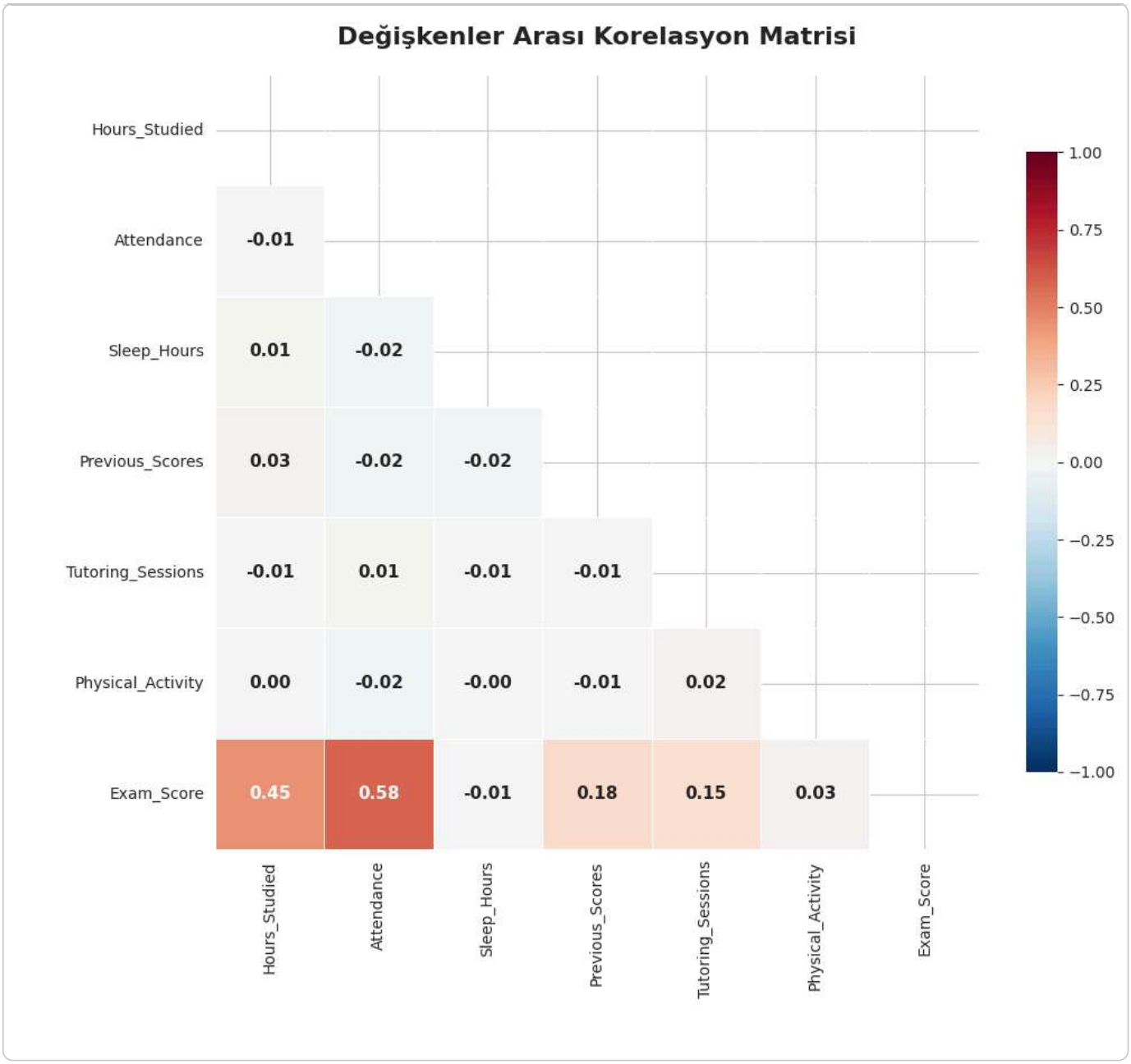
```
df.drop(index = df[df['Hours_Studied'] == 1].index , inplace = True ,axis = 0 )
plt.figure(figsize=(14, 7))

sns.boxplot(
    data=df,
    x='Hours_Studied',
    y='Exam_Score',
    palette='viridis',
    linewidth=1.5,
    fliersize=3
)

plt.title("Çalışma Saatinin Sınav Notuna Etkisi", fontsize=16, fontweight='bold', pad=20)
plt.xlabel("Çalışılan Saat", fontsize=12)
plt.ylabel("Sınav Notu", fontsize=12)

sns.despine(trim=True)
plt.grid(axis='y', alpha=0.3, linestyle='--')
plt.show()
```



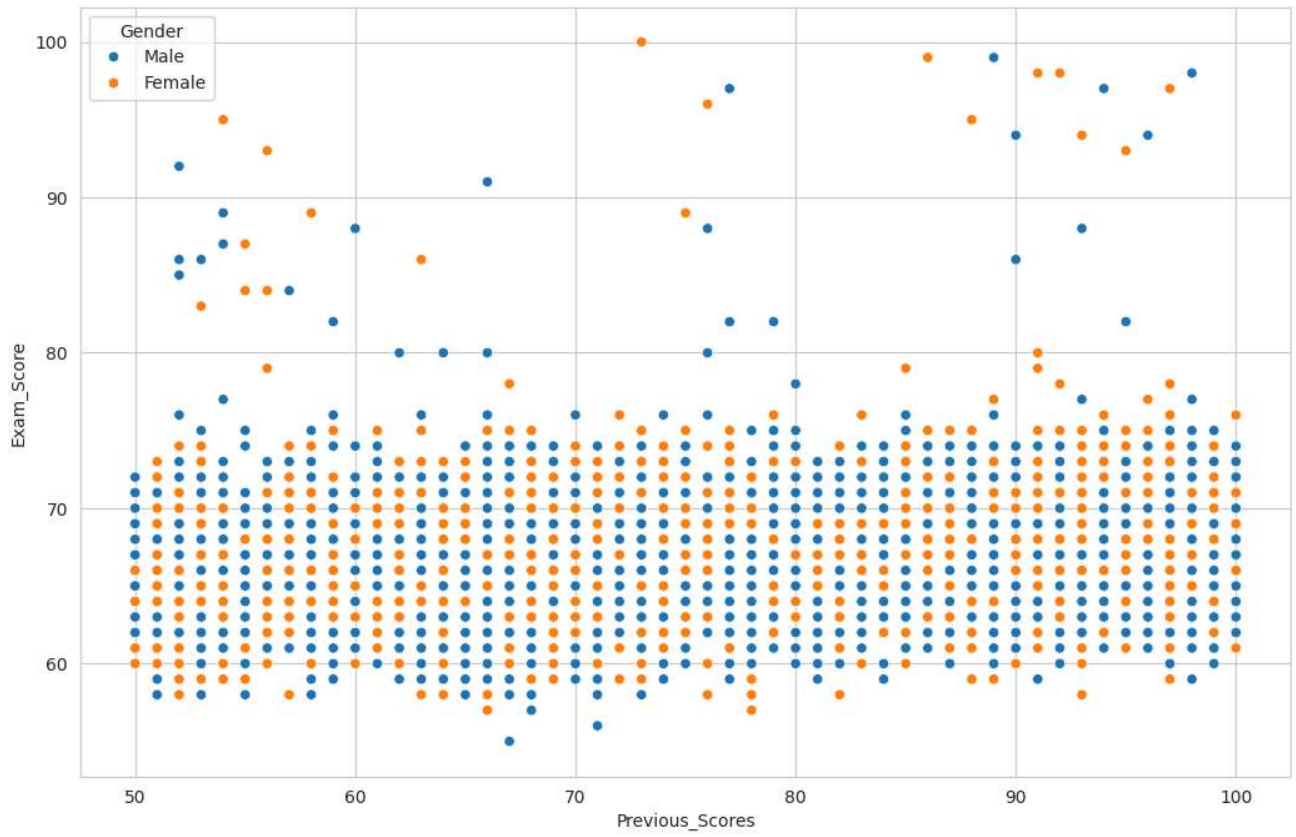

```
plt.figure(figsize=(12, 8))


sns.violinplot(
    data=df,
    x='Motivation_Level',
    y='Exam_Score',
    hue='Parental_Education_Level',
    palette='muted',
    split=False,
    inner="quartile",
    linewidth=1.5
)

plt.title("Motivasyon ve Aile Eğitiminin Başarıya Etkisi", fontsize=16, fontweight='bold')
plt.xlabel("Motivasyon Seviyesi", fontsize=12)
plt.ylabel("Sınav Notu", fontsize=12)
plt.legend(title='Aile Eğitimi', bbox_to_anchor=(1.05, 1), loc='upper left')

sns.despine()
plt.show()
```




```
corr = df.corr(numeric_only=True)


mask = np.triu(np.ones_like(corr, dtype=bool))

plt.figure(figsize=(11, 9))


sns.heatmap(
    corr,
    mask=mask,
    cmap='RdBu_r',
    center=0,
    vmax=1, vmin=-1,
    annot=True,
    fmt='.2f',
    linewidths=.5,
    cbar_kws={"shrink": .8},
    annot_kws={"size": 11, "weight": "bold"}
)

plt.title("Değişkenler Arası Korelasyon Matrisi", fontsize=16, fontweight='bold', pad=20)
plt.show()
```




```
plt.figure(figsize = (12.5 , 8 ))
sns.scatterplot(data = df , x = 'Previous_Scores' , y = 'Exam_Score' , hue = 'Gender')
plt.show()
```




```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder , OneHotEncoder
from sklearn.compose import ColumnTransformer
df['Effort_Score'] = df['Attendance'] * df['Hours_Studied']

df['Log_Hours'] = np.log1p(df['Hours_Studied'])

X = df.drop(['Exam_Score'] , axis = 1)
y = df['Exam_Score']



X_train , X_test , y_train , y_test = train_test_split(X , y , test_size = 0.3 , random_state = 15)

for col in ['Teacher_Quality' , 'Parental_Education_Level' , 'Distance_from_Home'] :
    X_train[col] = X_train[col].fillna(X_train[col].mode()[0])
    X_test[col] = X_test[col].fillna(X_train[col].mode()[0])


OrdinalEncoderColumn = ['Parental_Involvement' , 'Access_to_Resources' , 'Motivation_Level' , 'Family_Income' , 'Teache
arrangement = ['Low' , 'Medium' , 'High']
categories_list  = [arrangement] * len(OrdinalEncoderColumn)

OneHotColumn = ['Extracurricular_Activities' , 'Internet_Access' , 'School_Type' , 'Learning_Disabilities' , 'Gender']

order_parent = ['High School', 'College', 'Postgraduate']
order_distance = ['Near', 'Moderate', 'Far']
order_peer = ['Positive', 'Neutral', 'Negative']

single_ordinal_cols = ['Parental_Education_Level', 'Distance_from_Home', 'Peer_Influence']
single_ordinal_rules = [order_parent, order_distance, order_peer]

# 2
transformer = ColumnTransformer(transformers=[ ('onehot', OneHotEncoder(drop='first', dtype=int, sparse_output=False),


    ('ordinal_multi', OrdinalEncoder(dtype=int, categories=categories_list), OrdinalEncoderColumn),
```

```
    ('ordinal_singles', OrdinalEncoder(dtype=int, categories=single_ordinal_rules), single_ordinal_cols)

], remainder='passthrough', verbose_feature_names_out=False)

transformer.set_output(transform="pandas")

# 4
X_train = transformer.fit_transform(X_train)
X_test = transformer.transform(X_test)


X_train.head()
```

	Extracurricular_Activities_Yes	Internet_Access_Yes	School_Type_Public	Learning_Disabilities_Yes	Gender_Male	P
6386	1	1	1	0	1	
2019	1	1	1	0	1	
6015	1	1	1	0	0	
6435	1	1	1	0	1	
3708	0	1	1	0	1	

5 rows × 21 columns

```
from sklearn.ensemble import AdaBoostRegressor , GradientBoostingRegressor , RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from xgboost import XGBRegressor
import lightgbm as lgbm
```

```
models = {
    'ADA' : AdaBoostRegressor(),
    'GBR' : GradientBoostingRegressor(),
    'RFR' : RandomForestRegressor(),
    'XGB' : XGBRegressor(),
    'DTR' : DecisionTreeRegressor(),
    'LGBM' : lgbm.LGBMRegressor()
}
```

```
from sklearn.metrics import mean_absolute_error , mean_squared_error , r2_score
def calculate_metrics(true , pred) :
    r2 = r2_score(true , pred)
    mae = mean_absolute_error(true , pred)
    mse = mean_squared_error(true , pred)

    return r2 , mae , mse
```

```
results = []


for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    r2, mae, mse = calculate_metrics(y_test, y_pred)

    results.append({
        "Model": name,
        "R2 Score": r2,
        "MAE (Error)": mae,
        "MSE": mse
    })


results_df = pd.DataFrame(results)
```

```
results_df = results_df.sort_values(by="MAE (Error)", ascending=True)

results_df.style.background_gradient(cmap="Blues")
```

```
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.001057 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 485
[LightGBM] [Info] Number of data points in the train set: 4622, number of used features: 21
[LightGBM] [Info] Start training from score 67.155128
```

	Model	R2 Score	MAE (Error)	MSE
1	GBR	0.628006	0.851116	5.835427
5	LGBM	0.625285	0.858830	5.878116
3	XGB	0.594175	0.965364	6.366132
2	RFR	0.565288	1.163725	6.819283
4	DTR	0.014537	1.870772	15.458859
0	ADA	0.220239	2.320964	12.232031

```
from sklearn.model_selection import RandomizedSearchCV

lgbmRegressor = lgbm.LGBMRegressor(verbose = -1)

params = {
    'n_estimators' : [50 , 100 , 200 , 300],
    'learning_rate' : [0.1 , 0.2 , 0.4 , 0.65],
    'max_depth' : [-1 , 10 , 25 , 30 , None],
    'min_child_samples': [20, 50, 100],
    'num_leaves': [20, 31, 50, 80],
}

random = RandomizedSearchCV(estimator = lgbmRegressor , param_distributions = params , scoring = 'neg_mean_absolute_err

random.fit(X_train , y_train)

y_pred = random.predict(X_test)

r2 , mae , mse = calculate_metrics(y_test , y_pred)

print('\n R2 :  ' , r2)
print('\n MAE :  ' , mae )
print('\n MSE :  ' , mse)
```

```
Fitting 5 folds for each of 10 candidates, totalling 50 fits

 R2 :   0.6313186694336457

 MAE :   0.8283748229706689

 MSE :   5.78346898976397
```

```
model = GradientBoostingRegressor()
model.fit(X_train, y_train)
```

▾ GradientBoostingRegressor

```
GradientBoostingRegressor()
```

```
def predict_score(hours, attendance, sleep, previous, motivation, internet, family, teacher, school, peer, gender):

    import pandas as pd
    import numpy as np

    try:
        input_df = pd.DataFrame([{
            "Hours_Studied": float(hours),
            "Attendance": float(attendance),
            "Sleep_Hours": float(sleep),
```

```
                "Previous_Scores": float(previous),

                "Log_Hours": np.log1p(float(hours)),
                "Effort_Score": float(hours) * 2,
                "Gender": str(gender),

                "Motivation_Level": str(motivation),
                "Internet_Access": str(internet),
                "Family_Income": str(family),
                "Teacher_Quality": str(teacher),
                "School_Type": str(school),
                "Peer_Influence": str(peer),

                "Parental_Involvement": "medium",
                "Access_to_Resources": "medium",
                "Extracurricular_Activities": "Yes",
                "Tutoring_Sessions": 2,
                "Physical_Activity": 3,
                "Learning_Disabilities": "No",
                "Parental_Education_Level": "medium",
                "Distance_from_Home": "Near"
            }])

            print("INPUT DF:\n", input_df)   # 🔥 DEBUG

            input_transformed = transformer.transform(input_df)
            prediction = model.predict(input_transformed)[0]

            return f"Score: {round(prediction,2)}"

        except Exception as e:
            return f"ERROR: {str(e)}"
```

```
import gradio as gr

with gr.Blocks(theme=gr.themes.Soft()) as demo:

    gr.Markdown("# 🎓 Student Academic Performance Predictor")
    gr.Markdown("### Enter student details to predict exam score")

    with gr.Row():

        with gr.Column():
            gr.Markdown("### 📊 Academic Inputs")

            hours = gr.Number(label="📚 Hours Studied", value=5)
            attendance = gr.Number(label="🗓️ Attendance (%)", value=75)
            previous = gr.Number(label="📈 Previous Score", value=60)

        with gr.Column():
            gr.Markdown("### 🧠 Personal Inputs")

            sleep = gr.Slider(0, 12, label="Sleep Hours")

            motivation = gr.Dropdown(
                ["Low", "medium", "High"],
                label="Motivation Level"
            )

            internet = gr.Dropdown(
                ["Yes", "No"],
                label="Internet Access"
            )

            family = gr.Dropdown(
                ["Low", "medium", "High"],
                label="Family Income"
            )

            teacher = gr.Dropdown(
                ["Low", "medium", "High"],
                label="Teacher Quality"
```